

Direitos e Permissões de Arquivos no Linux

O Linux possui um sistema de permissões que controla **quem pode acessar um arquivo ou diretório e quais operações essa pessoa pode realizar**.

Esse mecanismo é fundamental para a segurança do sistema operacional, pois impede, por exemplo, que um usuário comum altere arquivos importantes do sistema ou leia arquivos particulares de outro usuário.

As três permissões fundamentais do Linux são:

```
r = read      = leitura
w = write     = escrita
x = execute   = execução
```

Essas permissões são aplicadas a três categorias:

```
u = user      = dono
g = group     = grupo
o = others    = outros usuários
```

Assim, as permissões normalmente seguem esta estrutura:

```
DONO | GRUPO | OUTROS
```

1. Visualizando as permissões

Podemos visualizar as permissões utilizando:

```
ls -l
```

Por exemplo:

```
-rw-r--r-- 1 marcelo alunos 1200 Aug 16 10:30 programa.go
```

A parte que nos interessa inicialmente é:

```
-rw-r--r--
```

Ela contém informações sobre o tipo e as permissões do arquivo.

Podemos separar:

```
- | rw- | r-- | r--
  ↓   ↓   ↓
  dono grupo outros
```

Portanto:

```
rw- → dono
r-- → grupo
r-- → outros
```

O dono pode ler e escrever.

O grupo pode apenas ler.

Os demais usuários também podem apenas ler.

2. O primeiro caractere

O primeiro caractere não representa propriamente uma permissão. Ele indica o **tipo do objeto**.

Por exemplo:

```
-rw-r--r--  
^
```

O - indica um arquivo comum.

Alguns exemplos são:

```
- = arquivo comum  
d = diretório  
l = link simbólico
```

Assim:

```
-rw-r--r--
```

é um arquivo.

Enquanto:

```
drwxr-xr-x
```

é um diretório.

3. As permissões r, w e x

As três permissões básicas são:

r: Read

Permite leitura.

```
r
```

Em um arquivo, significa que podemos visualizar seu conteúdo.

Por exemplo:

```
cat dados.txt
```

w: Write

Permite escrita.

```
w
```

Significa que podemos modificar o conteúdo do arquivo.

Por exemplo, editar:

```
nano dados.txt
```


Os valores são:

$r = 4$
 $w = 2$
 $x = 1$

Para descobrir o número correspondente às permissões, basta somar.

Por exemplo:

`rwX`

$r = 4$
 $w = 2$
 $x = 1$

$4 + 2 + 1 = 7$

Portanto:

`rwX = 7`

6. A tabela que você deve memorizar

A tabela completa é:

Número	Permissão	Significado
0	---	nenhuma permissão
1	--x	executar
2	-w-	escrever
3	-wx	escrever + executar
4	r--	ler
5	r-x	ler + executar
6	rw-	ler + escrever
7	rwX	ler + escrever + executar

Uma maneira simples de lembrar é:

LEITURA = 4
ESCRITA = 2
EXECUÇÃO = 1

Depois basta somar.

7. Entendendo a permissão 644

Uma permissão muito comum é:

`644`

Separe os números:

6 | 4 | 4
↓ | ↓ | ↓
U | G | O

Ou seja:

Dono	Grupo	Outros
6	4	4

Agora convertemos.

Dono = 6

4 + 2 = 6

r + w = rw-

Grupo = 4

4 = r--

Outros = 4

4 = r--

Portanto:

644

significa:

rw-r--r--

Ou:

Dono	→ ler e escrever
Grupo	→ somente ler
Outros	→ somente ler

8. Por que aparece 0644 em Go?

Em Go encontramos frequentemente códigos como:

```
arquivo, err := os.OpenFile(  
    "dados.txt",  
    os.O_APPEND|os.O_WRONLY|os.O_CREATE,  
    0644,  
)
```

O:

0644

está relacionado às permissões do arquivo quando ele precisa ser criado.

Nesse contexto, o zero inicial indica que estamos escrevendo o número utilizando **notação octal**.

Assim:

0644	
	outros
	grupo
	donos

O resultado pretendido é:

```
rw-r--r--
```

Ou seja:

```
Dono    → leitura + escrita  
Grupo   → leitura  
Outros  → leitura
```

É importante observar que, em sistemas Unix/Linux, a permissão efetiva de um arquivo recém-criado também pode ser afetada pela configuração `umask` do processo.

9. Alterando permissões com `chmod`

O comando mais conhecido para alterar permissões é:

```
chmod
```

Por exemplo:

```
chmod 644 dados.txt
```

Estamos determinando:

```
Dono    → rw-  
Grupo   → r--  
Outros  → r--
```

Podemos verificar:

```
ls -l dados.txt
```

Resultado semelhante a:

```
-rw-r--r-- dados.txt
```

10. Permissão 755

Outra permissão extremamente comum é:

```
755
```

Separando:

```
7 | 5 | 5
```

Temos:

```
7 = rwx  
5 = r-x  
5 = r-x
```

Portanto:

```
755 = rwxr-xr-x
```

Significa:

```
Dono    → ler + escrever + executar
```

Grupo → ler + executar
Outros → ler + executar

É comum encontrarmos 755 associado a diretórios e arquivos executáveis.

Por exemplo:

```
chmod 755 programa
```

11. Permissão 700

Considere:

```
chmod 700 programa
```

Temos:

```
7 = rwx  
0 = ---  
0 = ---
```

Portanto:

```
rwx-----
```

Somente o dono possui permissões.

Dono → ler + escrever + executar
Grupo → nada
Outros → nada

12. Permissão 600

Outra permissão importante é:

```
600
```

Significa:

```
6 = rw-  
0 = ---  
0 = ---
```

Resultado:

```
rw-----
```

Somente o dono pode ler e escrever.

É útil para arquivos que não devem ficar acessíveis aos demais usuários.

13. Permissão 777

Uma permissão muito conhecida é:

```
777
```

Isso significa:

```
7 = rwx
7 = rwx
7 = rwx
```

Portanto:

```
rwXrwxrwx
```

Todo mundo pode:

```
Dono    → ler + escrever + executar
Grupo   → ler + escrever + executar
Outros  → ler + escrever + executar
```

Apesar de resolver rapidamente alguns problemas de acesso, utilizar:

```
chmod 777 arquivo
```

indiscriminadamente é uma **má prática de segurança**.

A regra deve ser:

Conceda apenas as permissões realmente necessárias.

14. chmod também aceita letras

Não precisamos utilizar somente números.

Podemos utilizar:

```
u = user
g = group
o = others
a = all
```

E:

```
+ = adicionar
- = remover
= = definir exatamente
```

Por exemplo:

```
chmod u+x programa.sh
```

Significa:

```
u  → usuário/dono
+  → adicionar
x  → execução
```

Ou seja:

Adicione permissão de execução para o dono.

Outro exemplo:

```
chmod g+w dados.txt
```

Significa:

Adicione permissão de escrita para o grupo.

Para retirar:

```
chmod o-w dados.txt
```

Significa:

Retire a permissão de escrita dos outros usuários.

15. Dono e grupo

Além das permissões, arquivos Linux possuem um **dono** e um **grupo**.

Podemos verificar usando:

```
ls -l
```

Exemplo:

```
-rw-r--r-- 1 marcelo professores 1200 dados.txt
```

Temos:

```
marcelo      → dono
professores  → grupo
```

As permissões podem ser interpretadas como:

DONO	GRUPO	OUTROS
↓	↓	↓
marcelo	professores	demais usuários
rw-	r--	r--

16. Alterando o dono com chown

O comando:

```
chown
```

permite alterar o proprietário de um arquivo.

Por exemplo:

```
sudo chown joao dados.txt
```

Agora joao será o proprietário.

Também podemos alterar dono e grupo:

```
sudo chown joao:alunos dados.txt
```

Isso significa:

```
dono   = joao
grupo  = alunos
```

17. Alterando somente o grupo

Podemos utilizar:

```
chgrp
```

Por exemplo:

```
chgrp professores dados.txt
```

O arquivo passa a pertencer ao grupo `professores`, desde que o usuário tenha autorização para realizar essa alteração.

18. Permissões em diretórios

Em diretórios, `r`, `w` e `x` possuem significados um pouco diferentes dos arquivos comuns.

De forma simplificada:

`r` → permite listar os nomes existentes no diretório

`w` → permite alterar as entradas do diretório, como criar ou remover arquivos, quando as demais permissões necessárias também permitem

`x` → permite atravessar/acessar o diretório

Por exemplo:

```
drwxr-xr-x
```

Temos:

`d` → diretório

`rw` → dono

`r-x` → grupo

`r-x` → outros

Numericamente:

```
755
```

Por isso:

```
chmod 755 pasta
```

é muito comum.

19. Umask

Existe ainda um conceito importante chamado:

```
umask
```

Ele influencia as permissões finais utilizadas na criação de novos arquivos e diretórios.

Podemos verificar a configuração atual:

```
umask
```

Um resultado comum é:

0022

Isso ajuda a explicar por que um programa pode solicitar uma determinada permissão na criação de um arquivo e o arquivo acabar com permissões finais mais restritivas.

Portanto, no Go:

```
os.OpenFile(
    "dados.txt",
    os.O_CREATE|os.O_WRONLY,
    0666,
)
```

o 0666 representa o modo solicitado para a criação. A `umask` do sistema/processo pode remover algumas dessas permissões.

Com uma `umask` típica de 0022, por exemplo, o arquivo normalmente termina como:

0644

ou:

`rw-r--r--`

20. Exemplos para memorizar

Algumas permissões aparecem com bastante frequência:

600 = `rw-----` → somente o dono lê e escreve

644 = `rw-r--r--` → dono escreve, todos podem ler

700 = `rwX-----` → somente o dono possui acesso completo

755 = `rwXr-xr-x` → dono completo, outros leem/executam

777 = `rwXrwxrwx` → todos possuem acesso completo

Para um iniciante, vale memorizar principalmente:

4 = READ
2 = WRITE
1 = EXECUTE

Então:

7 = 4 + 2 + 1 = `rwX`
6 = 4 + 2 = `rw-`
5 = 4 + 1 = `r-X`
4 = = `r--`

A partir daí, fica fácil interpretar:

755

como:

7	5	5
↓	↓	↓
<code>rwX</code>	<code>r-X</code>	<code>r-X</code>

dono	grupo	outros

Conclusão

O sistema de permissões é um dos fundamentos da segurança do Linux. Cada arquivo ou diretório possui um proprietário, um grupo e permissões que determinam o que diferentes usuários podem fazer.

As três permissões fundamentais são:

r	=	leitura	=	4
w	=	escrita	=	2
x	=	execução	=	1

E são organizadas para:

DONO		GRUPO		OUTROS
------	--	-------	--	--------

Por isso, ao encontrar em um programa Go:

0644

podemos interpretar imediatamente:

6	4	4
↓	↓	↓
rw-	r--	r--
dono	grupo	outros

Entender essa lógica facilita tanto a administração de sistemas Linux quanto a programação em Go, principalmente ao trabalhar com pacotes como `os` e funções que criam ou manipulam arquivos.